MHA GRIEVANCE OPERATIONS

Integration and Data Flow Guide

Department workspace handoff for the citizen chatbot integration

Purpose. This guide explains how the current React, Express, and SQLite workspace creates staff access, scopes tickets to departments and wards, filters sensitive fields, updates cases, and accepts citizen intake. It is written so the chatbot teammate can connect to the existing API without sharing staff credentials or duplicating the department database.

Implementation truth. This document describes the checked-in code in server/index.js, server/db.js, src/App.tsx, src/types/grievance.ts, and scripts/rbac.integration.test.mjs. Unimplemented flows are labeled explicitly rather than presented as working features.

Project root: D:\hacknovate2026

1. System overview

The department workspace is a local full-stack prototype: React + TypeScript + Vite in the browser, Express JSON API on port 3001, and SQLite at data/mha-grievance.sqlite. Vite runs on port 5173 and proxies /api to the API. START_MHA_DASHBOARD.bat or npm run start:local starts both processes.

Layer	Current implementation	Integration consequence
Citizen chatbot	Owned by teammate; not in this repository.	Submit validated grievances to POST /api/intake/grievances. Do not call staff login.
Department UI	src/App.tsx; API-backed reactive state.	Browser is a client; API is authoritative for identity, permission, and scope.
HTTP API	Express 5, server/index.js; JSON; localhost:3001.	Local prototype. Deploy behind HTTPS and approved gateway before public use.
Persistence	SQLite, better-sqlite3, server/db.js.	One database stores staff, roles, wards, tickets, sessions, and activity.
Evidence/audio	Payload holds URL/metadata only.	No binary upload or object storage is implemented.

1.1 End-to-end request path

```
Citizen UI → chatbot/backend → POST /api/intake/grievances → validation + category/department mapping → tickets
+ activity_events → staff /api/bootstrap scoped by role → officer PATCH → SQLite state + audit event
```

Every staff API request is authorized server-side. Hiding a control in React is not access control. The server derives role and permissions from the authenticated account, not a role supplied by the request.

2. Department, hierarchy, and scope

Department is the horizontal data boundary. Each ticket has normalized division_id and category. Vertical access level identifies the kind of officer at that desk. Scope adds the geographic boundary. Effective visibility is the intersection of permission, division, ward assignments, and (for appellate users) appealed status.

```
Visible(ticket) = tickets:read AND matching scope AND (if Appellate Authority: appealStatus = APPEALED)
```

Access level	Required scope	Permission ceiling	Queue and responsibilities
Staff Admin (SUPER_ADMIN)	GLOBAL	read, resolve, reroute, users:manage, analytics:view	Global tickets and analytics; staff and role administration; protected system account.
Division Admin	DIVISION + divisionId	read, resolve, reroute, analytics:view	All tickets and category details for one department. No user management.
L1 Resolving Officer	WARD + department divisionId	read, resolve, reroute	Cases in the officer's assigned wards and assigned department.
Appellate Authority	DIVISION + divisionId	read, resolve, analytics:view	Only appealed cases in one department. No reroute permission.
Other Case Triage Officer	DIVISION + OTHER_TRIAGE	read, reroute	Only Other queue; routes OTHER case to a recognized

Access level	Required scope	Permission ceiling	Queue and responsibilities
			department; cannot resolve.
General ward-only role	WARD, no divisionId	role-defined subset; never users:manage	Assigned wards across divisions; category details/evidence are stripped.

toRole() removes stored permissions outside each vertical level ceiling before authorization. Custom GLOBAL roles are rejected; only protected ROLE-SUPER-ADMIN gets global visibility. Admin management APIs require that protected role ID, not merely users:manage.

2.1 Department and category catalog

divisionId	Desk label	category	Category details represented
CYBER_INFORMATION_SECURITY	Cyber & Information Security	CYBERCRIME	UTR/transaction, amount, fraud time, platform, suspect details
FOREIGNERS	Foreigners & Immigration	IMMIGRATION	Passport, visa/OCI reference, nationality, visa type, dates
DISASTER_MANAGEMENT	Disaster Management	DISASTER_RELIEF	Aadhaar, bank/IFSC, disaster, damage category
UT_ADMINISTRATION	Union Territory Administration	UT_ADMIN	UT, civic department, ward/zone, property ID
FREEDOM_FIGHTERS_REHABILITATION	Freedom Fighters & Rehabilitation	FREEDOM_FIGHTERS_REHABILITATION	PPO, beneficiary, bank branch/IFSC, sanction date
CENTRAL_ARMED_POLICE_FORCES	Central Armed Police Forces	CAPF_MISCONDUCT	Force, officer/badge, date/location, misconduct
NGO_FCRA	NGO & FCRA	FCRA_NGO	NGO, FCRA registration, state, violation
INTERNAL_SECURITY	Internal Security	INTERNAL_SECURITY	Nature, target location, suspect/organization
COUNTER_TERRORISM	Counter Terrorism & Counter Radicalization	COUNTER_TERRORISM	Platform, threat type, location/subject
LWE_NORTH_EAST	LWE & North East	LWE_NORTH_EAST	Region, complaint, police jurisdiction
BORDER_MANAGEMENT	Border Management	BORDER_MANAGEMENT	Border/state, village/post, issue category
JUDICIAL	Judicial Division	JUDICIAL	Prisoner/registration, prison, case/FIR, petition
POLICE	Police Divisions	POLICE	Officer/nominee, cadre, request type
CENTRE_STATE	Centre-State Division	CENTRE_STATE	States involved, dispute nature
INTERNAL_ADMINISTRATION	Internal Divisions	INTERNAL_DIVISIONS	Query type, reference/date
OTHER_TRIAGE	Other Cases Triage	OTHER	Narrative for triage; intake discards dynamic details

The UI groups related departments for display (such as CAPF & Police). The database and API retain their exact division IDs; integrations must send IDs, not labels.

2.2 Seeded role IDs

- ROLE-SUPER-ADMIN: protected Staff Admin, GLOBAL.
- ROLE-{DIVISION_ID}: system Division Admin for each of 15 regular divisions.
- ROLE-L1-{DIVISION_ID}: system L1 role with WARD scope and divisionId; its empty role ward list lets the user assignment define allowed wards.
- ROLE-APPEAL-{DIVISION_ID}: system Appellate role with DIVISION scope.

- ROLE-OTHER-CASE-TRIAGE: Other Cases department only.
- ROLE-WARD-DESK and ROLE-MULTI-WARD: generic ward-only roles. Current examples include WARD-101 and WARD-101/102.

Startup seeds hierarchy roles for all 15 non-Other divisions, plus Other triage, two generic ward roles, and Staff Admin. IDs are stable references; role names are editable and must not drive authorization.

The four matrix-defined vertical levels are Staff Admin/Super Admin, Division Admin, L1 Resolving Officer, and Appellate Authority. The implementation also has the special OTHER_CASE_TRIAGE access level, but only under the standalone Other Cases Triage department; it is not offered in regular departments.

3. Permission modules

Permission	Meaning	Server enforcement
tickets:read	Read queue and detail.	GET /api/tickets; bootstrap ticket list; ticket visibility before PATCH.
tickets:resolve	Resolve and write official action report.	PATCH status=RESOLVED and actionTakenReport.
tickets:reroute	Change triage status and route Other cases.	PATCH status transitions and routeToDivisionId.
users:manage	Staff/role administration capability label.	Admin routes additionally require protected Staff Admin role ID.
analytics:view	Scoped analytics and activity/audit.	GET /api/analytics/summary, /api/activity; bootstrap activities.

Level	Allowed modules
Staff Admin	All five
Division Admin	read, resolve, reroute, analytics
L1 Resolving Officer	read, resolve, reroute
Appellate Authority	read, resolve, analytics
Other Case Triage Officer	read, reroute

4. Scope and dynamic-field behavior

Role	Returned ticket rows	Dynamic data/evidence	Additional filter
Staff Admin	All divisions and wards	All payload fields	None
Division Admin	Exact divisionId	Own department details/evidence	None
L1 Resolving Officer	Department and user-assigned wards; intersection with role wardIds if nonempty	Own department details/evidence	None
Appellate Authority	Exact divisionId	Own department details/evidence	payload.appealStatus must equal APPEALED

Role	Returned ticket rows	Dynamic data/evidence	Additional filter
Other triage	OTHER_TRIAGE	details={} on intake; current response retains evidence	Can route only category OTHER
Generic ward role	Assigned ward IDs	Sensitive category fields/details/evidence stripped recursively	Cross-department by ward

ticketScope() applies SQL filters before serialization. ticketForRole() then removes sensitive keys for users who are not members of the ticket's owning division. A user in the owning division, including its L1 and appellate role, sees that division's dynamic details, subject to their ward/appeal queue filter.

Caution: Other Case Triage currently receives evidence metadata and routing retains evidence; the route removes payload.details. If policy requires no attachments at triage, change the role sanitizer and add a regression assertion. URLs are not fetched/proxied by this API.

5. SQLite schema

SQLite uses foreign keys, WAL journaling, and a 5-second busy timeout. JSON columns are serialized text. The API flattens payload_json fields into the returned ticket object. Relational columns remain the canonical query/index fields.

Table	Schema and purpose
roles	id PK; name UNIQUE; description; permissions_json; scope_type CHECK GLOBAL/DIVISION/WARD; division_id; ward_ids_json; is_system; access_level; created_at.
users	id PK; name; email UNIQUE NOCASE; mobile_number; password_hash; role_id FK; assigned_wards_json; is_active; avatar_initials; must_change_password; created_at; last_sign_in; password_reset_at.
wards	id PK; name; district; state; pincode_prefix; zone. Seed IDs WARD-101…WARD-108.
tickets	id PK; grievance_id UNIQUE; complainant_name; mobile_number; ward_id FK; division_id; category; status CHECK; sla_status CHECK; submission_timestamp; sla_deadline; payload_json; action_taken_report; created_at; updated_at. Indexed by ward, division, status.
sessions	token_hash PK; user_id FK ON DELETE CASCADE; expires_at; created_at.
activity_events	integer id PK; user_id FK; ticket_id FK; event_type; summary; details_json; created_at. Recent event query capped at 50.
app_migrations	id PK; applied_at. Tracks one-time migrations and legacy cleanup.

5.1 Ticket JSON payload

```
payload_json = {
  complainantProfile: { otpVerified, gender, email? },
  location: { state, district, pincode, completeAddress, coordinates },
  detectedLanguage, declarationOfTruth, audioFileUrl,
  originalRegionalText, translatedEnglishText, evidence: EvidenceAttachment[],
  details: <department-specific JSON>, appealStatus?
}
```

Keep grievanceId, wardId, divisionId, category, status, and timestamps in the relational columns; these drive filtering and indexes. There is no payload schemaVersion today.

5.2 TypeScript contracts and exact detail keys

src/types/grievance.ts defines Permission, ScopeType, AccessLevel, TicketStatus, SlaStatus, DivisionId, TicketCategory, Location, Ward, Role, User, TicketBase, EvidenceAttachment, ComplainantProfile, and a discriminated Ticket union. TypeScript types do not validate incoming citizen JSON at runtime.

Object	Fields
Role	id, name, description, permissions[], scopeType, optional divisionId, wardIds[], isSystem, accessLevel.
User	id, name, email, mobileNumber, roleId, assignedWards[], isActive, avatarInitials, optional lastSignIn/passwordResetAt. Password hash is never serialized.
Ward	id, name, district, state, pincodePrefix, optional zone.
Ticket common/base	id, grievanceId, complainantName, mobileNumber, complainantProfile, location, wardId, divisionId, category, status, slaStatus, submissionTimestamp, slaDeadline, detectedLanguage, declarationOfTruth, audioFileUrl, originalRegionalText, translatedEnglishText, evidence[], optional actionTakenReport.
Location	state, district, pincode, completeAddress, coordinates { latitude, longitude }.
ComplainantProfile	otpVerified, gender (FEMALE | MALE | NON_BINARY | NOT_SPECIFIED), optional email. API value is unverified caller metadata.
EvidenceAttachment	id, fileName, mimeType, url. Runtime intake currently does not enforce each key/type.

Category	divisionId	Ticket details keys in the current TypeScript union
CYBERCRIME	CYBER_INFORMATION_SECURITY	transactionId, amountLost:number, fraudDateTime, platformUsed, suspectDetails
IMMIGRATION	FOREIGNERS	passportNumber, applicationReferenceNumber, nationality, visaType, issueDate, expiryDate
DISASTER_RELIEF	DISASTER_MANAGEMENT	aadhaarNumber, bankAccountDetails { accountNumber, ifsc }, disasterName, damageCategory
UT_ADMIN	UT_ADMINISTRATION	specificUt, civicDepartment, wardOrZoneNumber, propertyId
FREEDOM_FIGHTERS_REHABILITATION	FREEDOM_FIGHTERS_REHABILITATION	ppoNumber, bankBranch, ifsc, freedomFighterName, sanctionDate
CAPF_MISCONDUCT	CENTRAL_ARMED_POLICE_FORCES	forceName, optional officerName/badgeNumber, incidentDate, incidentLocation, misconductType
FCRA_NGO	NGO_FCRA	ngoName, fcraRegistrationNumber, registrationState, violationNature
INTERNAL_SECURITY	INTERNAL_SECURITY	complaintNature, targetLocation, suspectOrOrganization
COUNTER_TERRORISM	COUNTER_TERRORISM	radicalizationPlatform, threatType, threatLocationOrSubject
LWE_NORTH_EAST	LWE_NORTH_EAST	regionOrVillage, complaintNature, policeStationJurisdiction
BORDER_MANAGEMENT	BORDER_MANAGEMENT	borderNameOrState, villageOrPost, issueCategory
JUDICIAL	JUDICIAL	prisonerName, prisonerRegistrationNumber, prisonName, courtCaseOrFirNumber, petitionNature
POLICE	POLICE	officerOrNomineeName, stateCadre, requestNature
CENTRE_STATE	CENTRE_STATE	statesInvolved:string[], disputeNature
INTERNAL_DIVISIONS	INTERNAL_ADMINISTRATION	queryType, referenceNumberOrDate
OTHER	OTHER_TRIAGE	Record<string, unknown> in TypeScript; API intake forces details={} and triage must not rely on category fields.

Some requirement labels are represented under normalized property names (for example transactionId covers UTR; applicationReferenceNumber covers visa/OCI reference; damageCategory covers loss category). Normalize chatbot input before POST; do not send both aliases.

6. Authentication and staff account flow

- Initial setup: GET /api/auth/setup-status; when no users exist, POST /api/auth/initial-admin creates the one Staff Admin account (password 8–200 chars, endpoint limit 5/hour).
- Onboarding: Staff Admin calls POST /api/users. The server validates account/role/ward assignment, bcrypt-hashes a random temporary password, sets must_change_password, and returns that password once.
- Login: POST /api/auth/login validates the active user, records last_sign_in and SIGN_IN, creates an opaque random token, stores only SHA-256(token) with 8-hour expiry, and returns token/user/effective role.
- Tab isolation: src/App.tsx stores token in sessionStorage key mha-tab-session and sends Authorization: Bearer token with credentials omitted. Each tab can sign in as a different account.
- First login/reset: only change-password or logout is allowed until password rotation. POST /api/auth/change-password accepts 8–200 chars and clears must_change_password.
- Logout revokes this token; suspension and password reset revoke all sessions for the affected user.

forgot-password only records an internal recovery request; no email is sent. recover-super-admin is rate-limited and disabled in production. Citizen users are not rows in the staff users table and must not use staff login.

7. Citizen intake contract

POST /api/intake/grievances is currently unauthenticated and rate-limited to 20 requests per IP per hour using an in-memory limiter. It validates basic identity/text lengths, the ward ID, and category/division pairing. It is suitable for localhost integration, not an internet-facing trusted boundary.

Request key	Required	Current behavior
complainantName	Yes	Trimmed, 2–120 characters.
mobileNumber	Yes	7–25 chars; no OTP is performed by this API.
wardId	Yes	Must exist in wards.
divisionId + category	Yes	Must match canonical map; OTHER requires OTHER_TRIAGE.
originalRegionalText + translatedEnglishText	Yes	Required; max 10,000 characters each.
location	No	If omitted derive from ward with 0,0 coordinates; provided object is not deep-validated.
detectedLanguage	No	Default Hindi; truncated to 40 chars.
details	No	Plain object; not deeply validated against category schema; discarded for OTHER.
audioFileUrl	No	Stored string; no upload API.
evidence	No	Up to 10 array entries stored; URL/item schema not fully validated.
complainantProfile / declarationOfTruth	No	Gender allow-list; OTP flag is caller-supplied metadata, not verification.

7.1 Example request

```
{
  "complainantName": "Synthetic Example Citizen", "mobileNumber": "+91 90000 00000",
  "wardId": "WARD-101", "divisionId": "CYBER_INFORMATION_SECURITY",
  "category": "CYBERCRIME", "detectedLanguage": "Hindi",
  "originalRegionalText": "[synthetic] Regional-language complaint",
  "translatedEnglishText": "[synthetic] English translation",
  "location": { "state":"Delhi", "district":"New Delhi", "pincode":"110001",
    "completeAddress":"Connaught Place", "coordinates":{"latitude":28.63,"longitude":77.22} },
  "details": { "transactionId":"SYN-UTR-001", "amountLost":1000,
    "fraudDateTime":"2026-10-01T10:00:00Z", "platformUsed":"Synthetic UPI",
    "suspectDetails":"synthetic@example" },
  "audioFileUrl":"https://approved-storage.example/object-id", "evidence":[]
}
```

Success returns HTTP 201 { ticket }. The API assigns TICKET UUID, MHA-YYYY-NNNN grievance ID, PENDING status, ON_TIME SLA, submission timestamp, and a fixed 21-day deadline, and records CITIZEN_INTAKE. Every repeated valid POST creates a new case; no idempotency key exists.

8. Triage routing and officer updates

8.1 Other Case routing

A permitted Other Case Triage Officer sends PATCH /api/tickets/:id with { routeToDivisionId }. The server checks the role level, tickets:reroute, source category OTHER, and destination ID. Destination cannot be OTHER_TRIAGE. The API updates divisionId and category to the destination's canonical category, removes payload.details, retains narrative and current evidence, changes updated_at, and records OTHER_CASE_ROUTED. The destination division queue can then see the case. No dedicated endpoint currently populates destination-specific details after routing.

8.2 Standard status/report update

PATCH /api/tickets/:id accepts { status, actionTakenReport }. The API checks ticket visibility before mutation; out-of-scope tickets return 404. Status enum: PENDING, UNDER_INVESTIGATION, RESOLVED. RESOLVED and report require tickets:resolve; transitions to/from pending or investigation require the appropriate reroute/resolve grant. Reports are trimmed and capped at 5,000 chars. Success updates SQLite and writes TICKET_UPDATED.

9. API endpoint map

Method/path	Authentication	Purpose
GET /api/health	Public	Service/SQLite health.
GET /api/auth/setup-status	Public	First-admin setup state.
POST /api/auth/initial-admin	Public, one-time	Bootstrap first Staff Admin.
POST /api/auth/login	Public	Staff sign-in; returns bearer token.
GET /api/auth/me	Bearer	Current user and role.
POST /api/auth/logout	Bearer	Revoke current session.

Method/path	Authentication	Purpose
POST /api/auth/change-password	Bearer	Password rotation.
POST /api/auth/forgot-password	Public	Records request; does not send email.
GET /api/bootstrap	Bearer	Scoped initial workspace bundle.
GET /api/tickets	Bearer + tickets:read	Scoped queue (max 500).
POST /api/intake/grievances	Public + IP limiter	Citizen complaint intake.
PATCH /api/tickets/:id	Bearer + action permission	Update ticket or route OTHER.
GET/POST/PATCH /api/roles	Staff Admin only	Role list/create/update.
GET/POST/PATCH /api/users	Staff Admin only	Directory/onboard/edit/suspend.
POST /api/users/:id/reset-password	Staff Admin only	Issue one-time password; revoke sessions.
GET /api/analytics/summary	Bearer + analytics:view	Scoped status/SLA counts.
GET /api/activity	Bearer + analytics:view	Scoped activity feed.

Development uses same-origin /api through Vite proxy. Direct browser cross-origin calls are not configured. If chatbot and API run on different origins, choose an allow-listed CORS/origin and CSRF design. Prefer chatbot backend-to-API calls; never put a service secret in browser JavaScript.

10. Constraints and non-implemented features

Area	Current constraint / truth	Integration consequence
Staff auth	Local staff accounts; bcrypt password; hashed bearer tokens; 8h expiry.	Citizen identity is separate; no public account login is provided.
Citizen auth	Intake has no authentication; OTP flag is caller-controlled.	Add gateway/service auth and verified identity before public exposure.
Scope	Server enforces permission, division, ward and appeal filters.	One shared DB, not physically isolated DB/encryption per division.
Validation	Category/division mapping and basic text/ward checks.	Deep per-category detail validation and strict allow-list missing.
Appeals	Queue depends on payload appealStatus=APPEALED.	No citizen appeal creation or trusted appeal status endpoint; tests set fixtures directly.
SLA	Fixed 21-day deadline; initially ON_TIME.	No scheduled SLA state recalculation job.
Audio/files	URL and metadata are stored.	No file upload, malware scan, signed URL proxy, or retention policy.
Citizen notifications	Not implemented.	No translation-back, SMS/WhatsApp/email, webhook, or delivery tracking.
Citizen tracking	Not implemented.	No public ticket lookup endpoint; do not expose staff tokens.
Audit	Activity events stored; recent query capped at 50.	SQLite is not immutable tamper-proof audit; no external log sink/history paging.
Demo data	RBAC tests use isolated temporary SQLite fixtures; local app DB is not seeded with	No synthetic test ticket persists in the normal app DB.

Area	Current constraint / truth	Integration consequence
	tickets.	

The existing README contains a stale statement that startup seeds 90 synthetic grievances. Current server/db.js does not seed complaint tickets, and /api/health reports syntheticData:false. The integration test fixtures are synthetic and removed after tests.

11. Chatbot integration checklist

- Use exact divisionId/category values from Section 2.1. Route uncertain classification to OTHER_TRIAGE + OTHER.
- Agree which chatbot backend sends the intake request. Do not embed staff credentials or service secrets in the citizen app.
- Do not trust caller-provided otpVerified; integrate a verified identity assertion if required.
- Only send category-specific fields appropriate to the selected department; add strict server-side schema validation before real PII.
- Upload media to approved storage and agree on durable, access-controlled references; current server does not upload or fetch bytes.
- Design idempotency before retrying requests; current duplicate POSTs create duplicate grievance records.
- After 201, present grievanceId and receipt in the chatbot. Notification delivery is currently the chatbot backend's responsibility.
- Agree future citizen tracking/status, appeals submission, translation notification, and appeal-state contracts; these are absent from current API.
- Use fictional test values until privacy, retention, encryption, access logging, and incident response requirements are agreed.

12. Test coverage and local commands

Test group	Assertions
All departments	Division Admin only sees its own category/data; seeded hierarchy roles exist.
All L1 tracks	Own department and assigned ward only.
All appellate tracks	Appealed cases only in own department.
Ward scopes	Single-/multi-ward isolation; dynamic fields/evidence stripped cross-department.
Staff Admin	All synthetic records/details and analytics visible.
Denied paths	Custom global role, illegal grant, role escalation, no-read, cross-scope update denied.
Lifecycle	L1 onboarding validates ward; temporary password rotation works.
Actions and triage	Ticket state/report update; Other cannot resolve; Other route clears details and reaches destination.

Local checks: double-click START_MHA_DASHBOARD.bat; open http://localhost:5173; check API at http://127.0.0.1:3001/api/health; run npm run build and npm run test:rbac.

The recorded audit run completed with 57 passing scenarios/subtests and zero failures. The test process starts a separate API on a free port with a temporary database and cleans it up. It does not seed the app's working database.

13. Source map

File	Contents
server/db.js	SQLite DDL/migrations, divisions, seeded roles, permission ceilings, serializers, audit query.
server/index.js	Authentication/session, scope functions, sensitive-field sanitizer, intake, mutations, staff APIs, analytics.
src/types/grievance.ts	Role/scope/ticket TypeScript interfaces and category-specific discriminated union.
src/App.tsx	Browser API helper, per-tab token, bootstrap, admin workflows, queue and ticket actions.
scripts/rbac.integration.test.mjs	Temporary synthetic data and end-to-end scenario tests.
package.json + vite.config.ts	Local start/build/test commands and /api development proxy.

This document is an implementation handoff for a local hackathon prototype, not a security certification or authorization to process real citizen data. The checked-in server code is authoritative if it differs from this guide.